

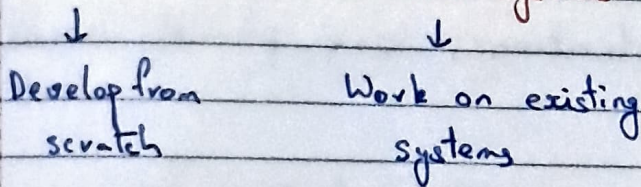
Design and Analysis of Software Systems

Development Cycle:- Dev Life Cycle [SDLC]

→ Determine whether the proposed sys is viable

- (i) Feasibility Analysis:- Req for proposal
- (ii) Requirements / Specifications:- What is needed, ~~How to~~
- (iii) Design:- How to do
- (iv) Implementation:-
- (v) Testing:- Static and Dynamic & Deployment
- (vi) ~~Main~~ Maintenance:-

Greenfield & Brownfield systems:-



Maintenance is max effort

Modules are never integrated in one shot, multiple steps
 During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model:- Types of SDLC, SDLC models

- (i) Traditional → Plan driven
- (ii) Agile → Dynamic

Processes must continually improve

De briefing - Planning & Execution

Convergence:- Estimates get more accurate as the rounds

Design and Analysis of Software Systems

Software

Development Cycle:- Dev Life Cycle [SDLC]

→ Determine whether the proposed sys is viable

- Modelling
- (i) Feasibility Analysis:- Req for proposal
 - (ii) Requirements / Specifications:- What is needed, ~~How to do~~
 - (iii) Design:- How to do
 - (iv) Implementation:-
 - (v) Testing:- Static and Dynamic & Deployment
 - (vi) Main Maintenance:-

Greenfield & Brownfield systems:-



Develop from
scratch



Work on existing
systems

Maintenance is max effort

Modules are never integrated in one shot, multiple steps
During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model:- Types of SDLC, SDLC models

- (i) Traditional → Plan driven
- (ii) Agile → Dynamic

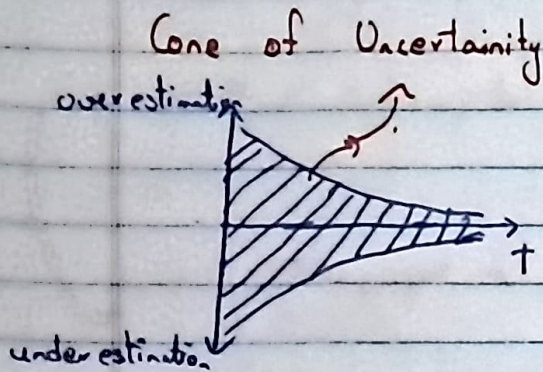
Processes must continually improve

De briefing - Planning & Execution

Convergence:- Estimates get more accurate as the rounds

progressed

Upfront Planning:-



Traditional:- No iterations, one planning session and no changes

Agile:- Many iterations, multiple shorter planning sessions between iterations

Team Dynamics & Self-organization

Leadership and Psychological Safety

Lean Principles

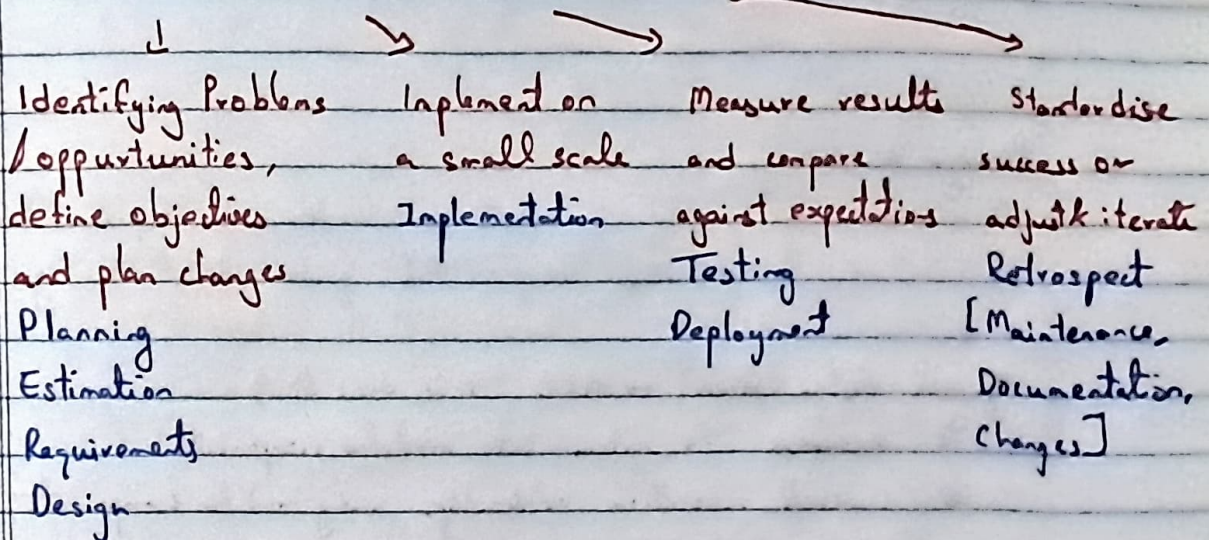
Work in Progress and The Definition of Done
when is the unit $\leftarrow$
actually done

Rework:- Starting from scratch once again
results
→ Technical Debt:-

PDCA - Continuous improvement mindset

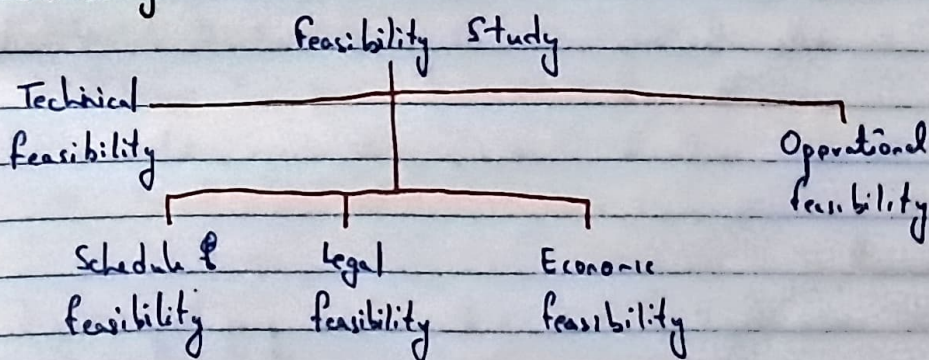
Alexander Deming

Plan - Do - Check - Act



1 PDCA cycle $\approx$ 1 iteration

Feasibility:-



Determine whether the proposed system is viable & worth pursuing

Requirements Engineering:-

Elicit, analyse, specify, validate & manage stakeholder needs so that the system built meets the intended purpose & constraints

SRS - Software Requirement Specifications

PRD - Product Requirement Document

Consists of

- ↳ req gathering & analysis (from diff pous)
- ↳ req specifications
- ↳ verification & validation
- ↳ req management & change control

Design :-

Transfer reqs into a robust, maintainable and implementable architecture & detailed designs that guide dev & testing.



High level design (Arch) → ADD (Arch desc doc)

- ↳ decompose the system into modules / components
- ↳ represent invocation relationships among modules / components
- ↳ specify constraints

Detailed Design → DD (Design Doc)

↳ UML

↳ data structures, class diagrams, interfaces & modules are designed

↳ API contracts, seqs, state diagrs, database schema.

Implement :-

- ↳ code front & back end in some prog lang

Testing :-

Test plan ---> Implementation

↓ !---> Design

Test !---> RE

Oracle

Dilbert!
PHD Comics!

7. V-Model

~~Rev~~

High

Maintenance:-

- Prevention → Defective prevention
- Correction → Bug fixes
- Perfection → improvements/enhancements
- Adaptive → Port software to new environments

Process Models:-

Traditional :- (plan driven)

- ↳ Waterfall model
- ↳ ~~Prototyping~~ Prototyping
- ↳ Evolutionary
- ↳ Spiral model et al

Agile :-

- ↳ eXtreme Programming (XP)
- ↳ Scrum
- ↳ Crystal
- ↳ Feature Driven Development (FDD)

Water fall model:- → Stability!

Seq process

Req., design, implementation, testing & maintenance need to be followed in that order

Each phase needs to be completed before moving on.

Challenges:-

- ↳ Heavy weight proc
- ↳ Doc intensive
- ↳ Minimal consumer involvement after req phase
- ↳ Less flexible design
- ↳ Big bang approach to coding & integration
- ↳ One shot delivery opportunity
- ↳ Limited opp for proc improvement

Good for systems with stable requirements

Prototyping Model → Clarity!

Before starting actual dev, working proto types are built to elicit custome. feedback.

Toy model

limited perf & efficiency.

Throwaway Throwaway
↳ Evolutionary.

Good for sys with unclear req, with focus on VI/UX

Morning!

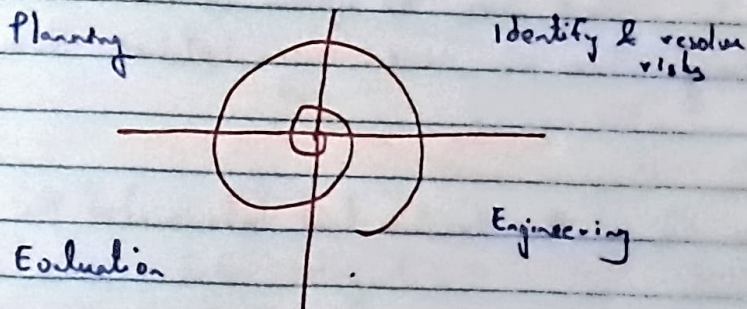
Evolutionary process model $\rightarrow$ Speed!

Sys is broken down into several modules which can be incrementally implemented & delivered.

Iterative incremental model $\rightarrow$ Modern Agile

Spiral Model $\rightarrow$ Safety!

- Strong emphasis on risk.
- Cont. cost feedback
- Sustainable for large, complex high risk systems
- Expensive but highly controlled.



Metaphor model as each loop of the spiral might be a process by itself.

Agile :-

1990 - 2000

- $\rightarrow$ Close collaboration b/w dev & business experts
- $\rightarrow$ 12 practices in the manifesto.
- $\rightarrow$ Maximise face-to-face communication (keep customer in the loop)
- $\rightarrow$ Self organising teams, everyone knows what they are doing and are owners themselves

Chow.

Many models under Agile

Incremental & Iterative in nature

Characteristics:-

- Short, but many quick releases
- Planning based on user stories
- Each iteration deals with all life cycle activities
- Testing ⇒ unit testing for deliverables; acceptance tests for each release
- Flexible Design ⇒ evolution vs big upfront effort
- Reflection after each cycle
- Several technical & customer focused presentation opportunities

User Stories:- What does the system do from the user's point of view? Many One or more user stories form a feature

Refactoring:- Change the structure (not behaviour) of the code

Dead Code:- Code that is very difficult to test / not possible to test at times

Open - Closed Principle:- Code is open only for extension, closed for modification

M

SOLID principles for OOP

Results in Technical Debt → debt incurred cause of bad coding practices.

Code Smell ← Bad cod (ing) practices

long methods violate single responsibility principle
↳ Modularize

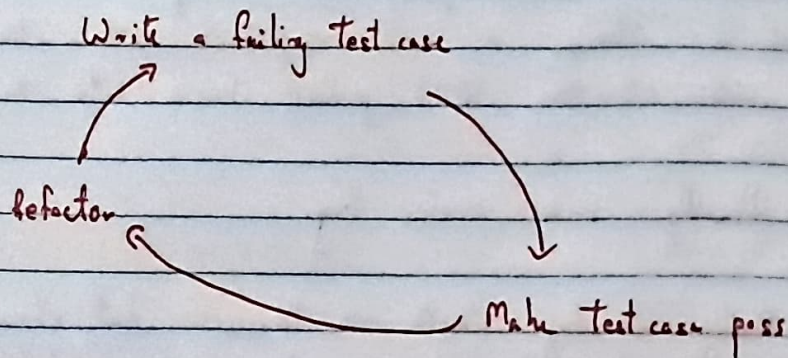
Test Driven Development

Write Tests First!

Test drives the development process

Write and Run Testcases.

Then write code to ensure testcases pass.



Source Code Integration

Big Bang Integration is bad

Source code (download from version control)

Resolve dependencies

Compile & Run

Manual Testing

Package & Deploy.

Continuous Integration → DevOps Development pl practice where devs merge changes into a shared repository frequently.

Commit → Build → Test → Feedback

Automated → Occurs continuously

Build Tools help with build process (MAVEN, ANT)

CD → Continuous Deployment (Automated)

↳ Continuous Delivery (Manual)

SCRUM Process:-

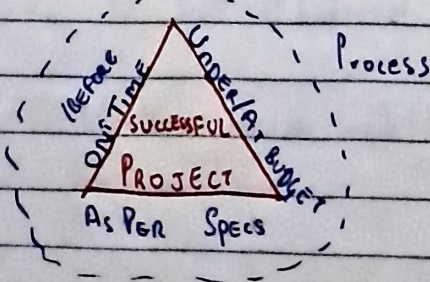
Backlog of User Stories

SCRUM master splits the backlog into multiple sprints.
If a story is not completed within the sprint,
and ~~at the~~ it is pushed back into the backlog.

Usually 2-4 weeks long.

Daily Standup Meets → Discuss what should be done,
what went well, etc to stay in the updated

Project Management



Facts need not be estimated.

For estimation

1) Always give a range, never a number

Experience from previous endeavours helps in making more accurate ~~lower~~ estimate -

person hour $\rightarrow$ • work done in 1 hour by 1 person

day = 8 person hours

week = 40 person hours

year = 2000 person hours

- 2) Always ask what the estimate will be used for.
- 3) Estimation $\neq$ Commitment
- 4) Measure, count and compute (try to atleast). Estimate only when necessary
- 5) Aggregate independent estimates
 $\hookrightarrow$ SLN $\rightarrow$ Planning Power.

Methods of Estimation:-

- i) Top-down
- ii) Bottom-up
- iii) Analogy
- iv) Expert Judgement
- v) Paid to Win
- vi) Parametric / Algorithmic

i) Function Point Analysis -

Software size measured by number & complexity of the fn it performs.

More methodical than LOC counts

a) Count no of fns per category

b) Estimate complexity of fns and apply a weighting multiplier

- ③ Compute on influence multiplier/value adjustment factor and apply
- ④ Results in in point total
- ⑤ Estimating effort & time.

Types of FPA

External Inputs

User/process inputs that creates or updates data.

Data-entry screen, API endpoints etc

External Outputs

Outputs sent outside the application that contain sales.

External Inquiries To

Online queries that retrieves data with little to no processing

Internal Logical Files

Application maintained logical data groups

External Interface Files

Read only logical data used by an application but stored externally

Assign Complexity Weights

For example

story $\approx$ 1 FP

oporo

	Input	Output
Low complexity	3 FP	4
Any	4	5
High	6	7

Refer Slides

Scheduling and Tracking

① Partition your project

Work Breakdown Structure

↓

Checklist of the work that must be accomplished to meet the project objectives

↳ Product / Process / Hybrid WBS

Fin engine

↓

↳ Both Product & Process

Req. Analysis, Design, Testing

Precedence Network

task

Shows the dependency of one task on the other.

↓

Identify the critical path → projects on it can have no slack.

↳ Slack Time $\leftarrow$ Max allowable delay for a non critical activity.

Requirements Engineering

↓

statement that specifies what a system must do, what qualities it must possess to satisfy stakeholder needs

Main reason why projects fail

Traceability:- Very important

Root Cause Analysis for bugs

Is it possible to be absolutely certain of the verity of having correct requirements

Open systems:- Req keep changing (maybe unconstrained)

Closed systems:- (Very) Fixed requirements

We must define the scope by having a more precise problem.

Learn to say No

Scope Creep $\leftarrow$ Scope keeps increasing

Req must
be determined

Clients produced
req

New
Development

A

B

Evolution
of an existing
system

C

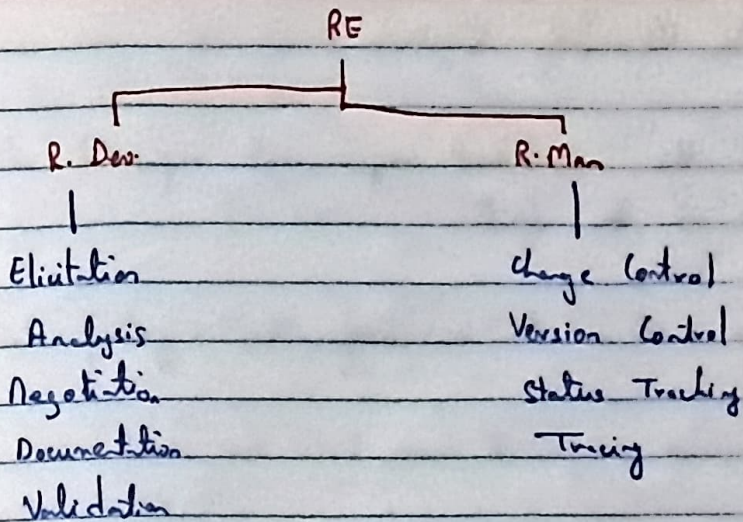
D

$\rightarrow$ Clearly stating reduces scope creep

① Functional Requirements $\leftarrow$ Features / what should the system do $\rightarrow$ expressed as functions

② Non-functional Requirements $\leftarrow$ qualities / characteristics of the system which cannot be expressed as fns

$\rightarrow$ Clear specific specifications can help measure how well the system is doing what it is supposed to do.



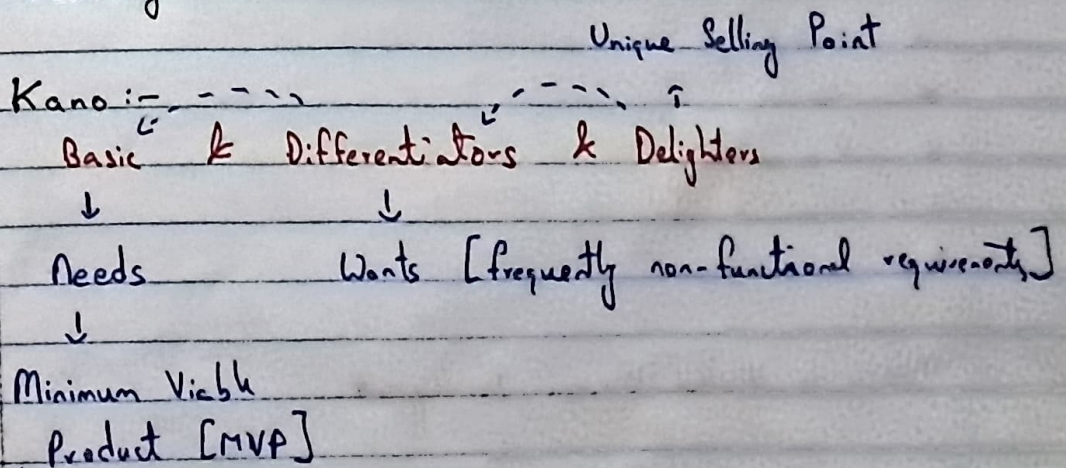
MVP :- Minimum Viable Requirements Product

MoSCoW method :- Must have, Should have, Could have, Won't have → Misuse cases opposite of user stories

Other methods

- (i) Interviews
- (ii) Observation
- (iii) Workshop / Joint Application Development

Quality Control :-



Good requirements engineering reduces rework.

Requirements Quality Control

Check if the identified requirements represent all expectations of the client

Validation :- Am I building the right product?

Verification :- Am I building the product right?

- ↳ Clearly understand the user req
- ↳ Ensure quality characteristics are met.

- * Correctness
- * Completeness
- * Unambiguity
- * Feasibility
- * Necessity
- * Priority
- * Consistency
- * Concise
- * Traceability
- * Standard Conformity
- * Verifiability

MDD ← Model Driven Development

1 SRS document

One way of specification engineering

Could use propositional logic, but we end up using English

Our main requirements are functional

Non-fn requirements are tied directly or indirectly to fn reqs

- ↳ system level reqs
- ↳ tied to multiple fn reqs

Transitioning from Problem to Solution

Wants / Needs → what must the system do? → How should the system do it?

Requirements Modelling :-

Transition from textual requirements → visual design

Helps in validating requirements.

(i)

Use case Case Modelling:-

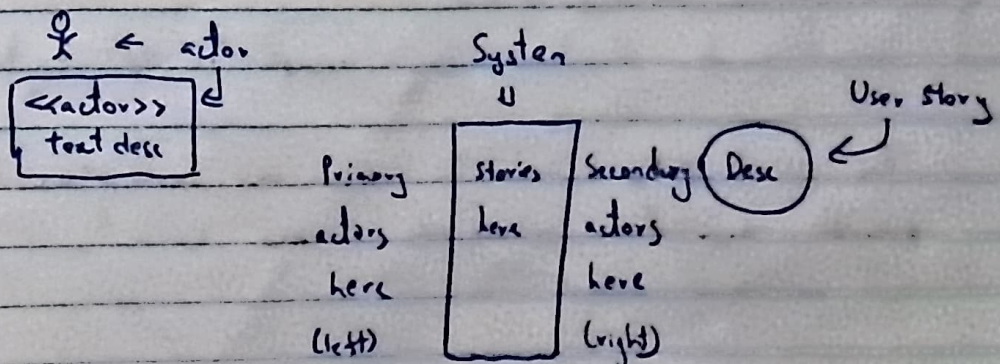
↳ a seq of actions including variants that a system or any other entity can perform, interacting with the actors of the system

Focuses on what to do not how to do it.

Actor:- Initiates specific behaviours of the system. Role played by entities interacting with the system.

↳ system, subsystems, classes

System/Subject Boundary:- Boundary b/w subject & actors



- (i) Identify actors $\leftarrow$ External & internal
- (ii) Identify goals $\leftarrow$ What does the user want to achieve
- (iii) Define use cases $\leftarrow$ One per goal.

There is a use case template

Post conditions $\leftarrow$ Implementation of behaviour

Scenario:- Specific seq or path in a use case

Use case instance:- execution of scenario

Per system design.

We must now define the use cases in detail

Needs to contain certain basic information.

Use case Name

Description

Actors

Pre-conditions

Main flow

Alt. flow

Post conditions

Conceptual Model:-

Identify & structure key concepts in the system's domain

The N & NPs in the Use Case description list the possible candidate conceptual classes

Not software classes

primitive
 ↑
 actual state of specific
 candidate class
 ↑
 Identify attributes & relationships b/w classes
 Validate into for completeness & realism

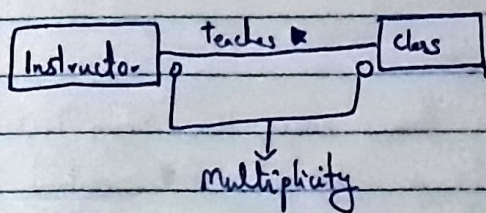
Association; direction reading arrow
 ?
 ↳ no meaning sans direction of reading the label

Private $\Leftarrow -$

Public $\Leftarrow +$

Protected $\Leftarrow \#$

$[a..b] \Leftarrow$ at least a , at most b occurrences



UML $\Leftarrow$ Unified Modelling Language

Testing - Guest Lecture

Test:- Activity in which a system or a component is executed under specified conditions, the results are observed & recorded & an evaluation is made of some aspect of the system

Test Case:- A set of preconditions, inputs & expected results, developed to drive the execution of the system.

Exhaustive Testing:- Test every possible input to every component of your system.

Testing based on equivalence classes:-

Class Group of like objects

Equivalence Classes:- Classes that are (expected to be) treated equally by the system

If we assume that every input in an equivalence class is treated the same, then testing one input in the group is sufficient

But this fails in case of inputs with limited scope (broken, etc.)

We also need to test combinations, boundaries, 0 values [edge cases] and also perform some random tests.

Based on coverage:-

- 1) Statement coverage (has every line been tested once)
- 2) Condition coverage (have all paths of conditions been covered)
- 3) Decision coverage (more general than condition)
- 4) Branch coverage (Decision + implicit conditions)
- 5) Modified Condition Decision Coverage (Every input is toggled independently)

Form Testing:-

Testing should always gather evidence, and testing should not be done by the same team as the developer

Why Test?

↳ Defects

↳ Evidence for conformance to

↳ Specs

↳ Stds

↳ Logs

↳ Safe & Unsafe scenarios to use a product

↳ Assess the quality

↳ Support for decision making

Types of testing:-

- ↳ Unit
- ↳ Integration
- ↳ System
- ↳ Acceptance

Black box & White box



You only know
the requirements to be
met

You know what to test exactly
All levels supported

For system level

Design Level

A

B

C

D



Fail only
once in
1 bil.

Statement

All elements coverage is sufficient
in a phase

∃ a format to write a test case.

Contains

- ↳ a unique id #
- ↳ objective
- ↳ priority
- ↳ traceability
- ↳ pre condition #
- ↳ inputs #
- ↳ expected outputs #

Test procedure:- Multiple sets of test cases.

Collection of test cases that need to be executed in a particular

order.

Mutation Testing :- To check for bugs in the very design of the system